

CS 4391 Term Project Report: Scene Recognition

Han Nguyen

1. Introduction

This project implements a scene recognition pipeline that classifies images into four categories: bedroom, desert, landscape, and rainforest. We compare four different approaches, ranging from simple pixel comparison to a convolutional neural network, to understand how feature representation affects classification accuracy.

The dataset contains 600 training images (150 per category) and 200 test images (50 per category).

2. Implementation

2.1 Pre-processing

Each training image was converted to grayscale to reduce complexity from 3 color channels down to 1. We then adjusted brightness: if the average pixel intensity fell below 0.4 (on a 0 to 1 scale), we increased it using OpenCV's `convertScaleAbs` with `alpha=1.5` and `beta=30`; if it exceeded 0.6, we decreased it with `alpha=0.7` and `beta=-30`. This helps normalize lighting differences across images so the classifiers receive more consistent input.

Each image was then resized to two versions: 200x200 (used for SIFT, Histogram, and CNN methods) and 50x50 (used for raw pixel classification).

2.2 SIFT Feature Extraction (Bag of Visual Words)

SIFT (Scale-Invariant Feature Transform) detects keypoints in an image, such as corners and edges, and describes each one with a 128-dimensional vector. Since different images produce different numbers of keypoints, we used a Bag of Visual Words (BoVW) approach to create a fixed-size representation:

1. Ran SIFT on all 600 training images, collecting 206,521 total descriptors from 598 images (2 images had no detectable keypoints).
2. Clustered these descriptors into 50 groups using K-Means. Each cluster center represents a “visual word.”
3. For each image, counted how many of its descriptors belong to each cluster, producing a normalized 50-dimensional histogram.

2.3 Histogram Feature Extraction

For each 200x200 grayscale image, we computed a 256-bin intensity histogram counting how many pixels fall at each brightness level (0 to 255). The histogram was normalized so values sum to 1. This produces a simple summary of an image's overall brightness distribution.

2.4 Classification Methods

Method A: k-NN on Raw Pixels. Each 50x50 image was flattened into a 2,500-dimensional vector. To classify a test image, we computed the Euclidean distance to every training image and assigned the label of the closest match (1-Nearest Neighbor).

Method B: k-NN on SIFT BoVW. Same nearest neighbor approach, but using the 50-dimensional BoVW histograms instead of raw pixels. Test images had their SIFT features extracted and mapped to the same vocabulary built during training.

Method C: k-NN on Histogram. Same nearest neighbor approach, using the 256-dimensional intensity histograms.

Method D: CNN. A convolutional neural network was trained on the 200x200 grayscale images. The architecture consisted of three convolutional layers (16, 32, and 64 filters) each followed by ReLU activation and 2x2 max pooling, then two fully connected layers (128 units, then 4 output classes). Training used the Adam optimizer with a learning rate of 0.001 and cross-entropy loss for 30 epochs with batch size 16. The model reached 100% training accuracy by epoch 20.

3. Results

3.1 Overall Summary

Each classifier was tested on all 200 test images (50 per category). The table below shows overall accuracy.

Method	Correct	Total	Accuracy
A: k-NN Raw Pixels	77	200	38.5%
B: k-NN SIFT BoVW	126	200	63.0%
C: k-NN Histogram	105	200	52.5%
D: CNN	127	200	63.5%

3.2 Method A: k-NN on Raw 50x50 Pixels

Per category correct classification (77/200 = 38.5%):

Category	Correct	Total	Accuracy
bedroom	4	50	8.0%
desert	45	50	90.0%
landscape	21	50	42.0%
rainforest	7	50	14.0%

Confusion matrix (rows = actual, columns = predicted):

	bedroom	desert	landscape	rainforest
bedroom	4	37	4	5
desert	1	45	3	1
landscape	0	28	21	1
rainforest	1	26	16	7

False positives and false negatives:

Category	FP	FP Rate	FN	FN Rate
bedroom	2	1.3%	46	92.0%
desert	91	60.7%	5	10.0%
landscape	23	15.3%	29	58.0%
rainforest	7	4.7%	43	86.0%

3.3 Method B: k-NN on SIFT BoVW

Per category correct classification (126/200 = 63.0%):

Category	Correct	Total	Accuracy
bedroom	29	50	58.0%
desert	25	50	50.0%
landscape	30	50	60.0%
rainforest	42	50	84.0%

Confusion matrix (rows = actual, columns = predicted):

	bedroom	desert	landscape	rainforest
bedroom	29	3	10	8
desert	13	25	8	4
landscape	4	6	30	10
rainforest	2	0	6	42

False positives and false negatives:

Category	FP	FP Rate	FN	FN Rate
bedroom	19	12.7%	21	42.0%
desert	9	6.0%	25	50.0%

Category	FP	FP Rate	FN	FN Rate
landscape	24	16.0%	20	40.0%
rainforest	22	14.7%	8	16.0%

3.4 Method C: k-NN on Histogram

Per category correct classification ($105/200 = 52.5\%$):

Category	Correct	Total	Accuracy
bedroom	25	50	50.0%
desert	20	50	40.0%
landscape	26	50	52.0%
rainforest	34	50	68.0%

Confusion matrix (rows = actual, columns = predicted):

	bedroom	desert	landscape	rainforest
bedroom	25	9	15	1
desert	19	20	10	1
landscape	10	5	26	9
rainforest	4	0	12	34

False positives and false negatives:

Category	FP	FP Rate	FN	FN Rate
bedroom	33	22.0%	25	50.0%
desert	14	9.3%	30	60.0%
landscape	37	24.7%	24	48.0%
rainforest	11	7.3%	16	32.0%

3.5 Method D: CNN

Per category correct classification ($127/200 = 63.5\%$):

Category	Correct	Total	Accuracy
bedroom	24	50	48.0%
desert	34	50	68.0%
landscape	27	50	54.0%
rainforest	42	50	84.0%

Confusion matrix (rows = actual, columns = predicted):

	bedroom	desert	landscape	rainforest
bedroom	24	10	9	7
desert	7	34	9	0
landscape	5	11	27	7
rainforest	3	0	5	42

False positives and false negatives:

Category	FP	FP Rate	FN	FN Rate
bedroom	15	10.0%	26	52.0%
desert	21	14.0%	16	32.0%
landscape	23	15.3%	23	46.0%
rainforest	14	9.3%	8	16.0%

4. Analysis and Discussion

4.1 Why raw pixels performed worst (38.5%)

Raw pixel comparison produced highly skewed predictions: 90% of desert images were classified correctly, but only 8% of bedrooms and 14% of rainforests were. Looking at the confusion matrix, the classifier predicted “desert” for 37 of 50 bedrooms, 28 of 50 landscapes, and 26 of 50 rainforests. Pixel by pixel Euclidean distance favors images with uniform, mid-range brightness (desert scenes), causing nearly everything to be pulled toward that class. This method is also extremely sensitive to spatial shifts: if a bed appears in a slightly different position between two bedroom photos, their pixel vectors can look entirely different.

A major contributing factor is the 50x50 resolution. Downsampling from the original ~350x250 to just 2,500 pixels discards nearly all fine detail such as textures, edges, and object boundaries. What remains is essentially a blurry, low resolution thumbnail dominated by average color and coarse brightness patterns. At this resolution, images of very different scenes can end up looking numerically similar (especially if they share an average brightness), which is exactly why most test images were pulled toward the dominant desert class. Using a higher resolution would preserve more information, but raw pixel comparison would still be a weak approach overall.

4.2 Why histograms did moderately better (52.5%)

Converting to intensity histograms removes spatial information and focuses on brightness distribution. This improved predictions across all categories, especially

rainforest (68%) which has a distinctive dark, low-intensity profile. However, histograms confused bedroom with landscape frequently (15 of 50 bedrooms misclassified as landscapes, 10 of 50 landscapes as bedrooms) because these categories can share similar mid-range brightness distributions. Deserts still dominated some predictions but less aggressively than with raw pixels.

The fundamental limitation of histograms is that they discard both spatial layout and color. A bright kitchen and a bright beach look identical to a grayscale intensity histogram. Extensions such as spatial pyramid histograms (computing histograms over image sub-regions) or color histograms would preserve more information. Since our preprocessing required grayscale input, only intensity histograms were used.

4.3 Why SIFT BoVW performed well (63.0%)

SIFT captures local structural patterns (edges, corners, textures) which are more informative than raw pixels or brightness alone. Rainforest achieved 84% accuracy, likely because dense foliage produces distinctive texture patterns that map to specific visual words. The error distribution was more balanced across categories compared to Method A. One weakness: deserts were only 50% correct, partly because smooth sand surfaces generate fewer distinctive SIFT keypoints, so the BoVW representation is less discriminative for them.

The vocabulary size ($k = 50$) also affects the result. Too few visual words means every image's histogram looks similar and the classifier cannot distinguish categories. Too many produces very sparse histograms that overfit to training specifics. Our choice of 50 is a reasonable middle ground, but tuning k (or using spatial pyramid matching to preserve some location information) could improve accuracy further.

4.4 Why the CNN was best overall (63.5%)

The CNN achieved the highest overall accuracy and outperformed SIFT on desert classification (68% versus 50%) because it can learn features suited to smooth textures that SIFT struggles with. It tied SIFT on rainforest accuracy (84%). However, it reached 100% training accuracy by epoch 20 while only achieving 63.5% on the test set, indicating significant overfitting. With only 150 training images per class, the network memorized specific training examples rather than learning fully generalizable features.

Several factors limited the CNN's accuracy:

- **No regularization.** The architecture has no dropout layers and the optimizer has no weight decay, so there is nothing discouraging the network from memorizing training examples.
- **No data augmentation.** Random flips, rotations, or crops would effectively expand the training set and force the network to learn features invariant to small changes.

- **Grayscale input.** The required grayscale preprocessing discards color information that would naturally help a CNN separate green rainforest from yellow desert, for example.
- **Simple architecture.** The three convolutional layers are enough to learn basic features but lack the depth of modern architectures. A pre-trained backbone (such as ResNet) fine-tuned on this dataset would almost certainly outperform our from-scratch network.

All of these improvements were outside the spec, which required grayscale preprocessing and left the architecture open-ended but did not mandate regularization or augmentation.

4.5 Which categories were hardest

Bedroom was consistently the hardest category across all methods (8% to 58% accuracy). Indoor scenes vary widely in furniture, layout, camera angle, and lighting, making them harder to characterize with any single feature type. Rainforest was the easiest for the feature-based methods (68% to 84%) because dense vegetation produces a consistent texture and dark color signature. Desert classification depended heavily on the method: 90% for raw pixels (due to classifier bias toward bright uniform scenes) but only 40% to 68% for more structured features.

4.6 Key factors affecting accuracy

- **Input resolution.** The 50x50 input for Method A discards most visual detail, which severely limits how much information the classifier has to work with. Methods B, C, and D all use the 200x200 version of the image (16 times more pixels), giving them access to much richer information.
- **Training set size.** 150 images per category is small, especially for CNN training, and contributes to overfitting.
- **Feature representation matters more than classifier complexity.** SIFT BoVW with a simple 1-NN classifier (63.0%) nearly matched the CNN (63.5%). Engineering good features can compensate for a simple classifier.
- **Different features favor different categories.** Raw pixels favor bright uniform scenes; histograms favor scenes with extreme brightness; SIFT favors textured scenes; the CNN balances better across categories but overfits.
- **The gap between SIFT (63.0%) and raw pixels (38.5%)** highlights why feature engineering was central to computer vision before deep learning. Choosing what information to extract from an image has a larger impact than the classification algorithm itself.